



**Department of Electrical,
Computer, & Biomedical Engineering**
Faculty of Engineering & Architectural Science

Course Title:	Distributed and Cloud Computing
Course Number:	COE 892
Semester/Year (e.g.F2016)	W2026

Instructor:	Dr. Muhammad Jaseemuddin
--------------------	--------------------------

<i>Assignment/Lab Number:</i>	4-5
<i>Assignment/Lab Title:</i>	FastAPI and Containers

<i>Submission Date:</i>	Apr 3, 2026
<i>Due Date:</i>	Apr 3, 2026

Student LAST Name	Student FIRST Name	Student Number	Section	Signature*
Pathirana	Chanuth	501107354	01	C.P

*By signing above you attest that you have contributed to this written lab report and confirm that all work you have contributed to this lab report is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: <http://www.ryerson.ca/senate/current/pol60.pdf>

Introduction

The objective of this 2-in-1 lab is to modify Lab 2 by developing two programs using FastAPI to simulate RESTful communication between a server and an operator. The server, developed as a FastAPI HTTP application, provides endpoints that allow the operator to create, modify, delete, dispatch, and control rovers in real time. For this lab, the server and rovers are considered as a single module, with the server handling all rover-related operations upon various requests, such as simulating rover movements, demining, and other tasks before returning these results to the operator. All communication between the server and operator is conducted in JSON format, which ensures structured data exchange. Additionally, an Azure account is created to deploy the final product.

Implementation

Part 1: Create the server

This server application was implemented using FastAPI to simulate a rover-based demining system that supports RESTful communication between an operator and the system. The application maintains internal data structures for the map, mines, and rovers, where the map is represented as a 2D grid loaded dynamically from a map.txt file and updated whenever mines are created, modified, or deleted. The server exposes multiple endpoints that allow the operator to manage the map dimensions, perform full CRUD operations on mines, and create, update, dispatch, and delete rovers. Each rover executes a sequence of commands (such as move, turn, and dig) based on logic adapted from Lab 2, where its movement is simulated step-by-step while checking for boundaries and mine collisions. The execution result includes the rover's final status (Finished or Eliminated), its final position, the list of executed commands, and a visual path of traversal. Proper error handling is implemented using HTTP status codes and descriptive messages for invalid requests, such as accessing non-existent resources or providing out-of-bounds coordinates. Overall, the server integrates simulation logic with RESTful APIs, enabling dynamic interaction and real-time control of rovers through structured JSON communication.

```
from fastapi import FastAPI, HTTPException, Path, Query
from pydantic import BaseModel
from typing import List, Optional, Dict, Any
import copy
import uvicorn

app = FastAPI(title="Rover Demining Server")

# -----
# DATA
# -----
MAP_ROWS, MAP_COLS = 10, 10
land_map = [[0 for _ in range(MAP_COLS)] for _ in range(MAP_ROWS)]
mines: List[Dict[str, Any]] = []
```

```

rovers: Dict[int, Dict[str, Any]] = {}

# -----
# MODELS
# -----
class MineCreate(BaseModel):
    serial: str
    x: int
    y: int

class MineUpdate(BaseModel):
    serial: Optional[str] = None
    x: Optional[int] = None
    y: Optional[int] = None

class RoverCreate(BaseModel):
    commands: str

# -----
# ROVER LOGIC (Lab 2)
# -----
DIRECTIONS = ['N', 'E', 'S', 'W']
MOVE = {'N': (-1, 0), 'E': (0, 1), 'S': (1, 0), 'W': (0, -1)}

def turn_left(direction):
    return DIRECTIONS[(DIRECTIONS.index(direction) - 1) % 4]

def turn_right(direction):
    return DIRECTIONS[(DIRECTIONS.index(direction) + 1) % 4]

def execute_rover_commands(commands: str):
    rover_land = copy.deepcopy(land_map)
    x, y = 0, 0
    direction = 'S'
    alive = True

    path = [['.' for _ in range(MAP_COLS)] for _ in range(MAP_ROWS)]
    path[x][y] = '*'

    executed = []

    for cmd in commands:
        if not alive:
            break

        if cmd == 'L':
            direction = turn_left(direction)

        elif cmd == 'R':

```

```

        direction = turn_right(direction)

    elif cmd == 'D':
        if rover_land[x][y] == 1:
            rover_land[x][y] = 0

    elif cmd == 'M':
        dx, dy = MOVE[direction]
        nx, ny = x + dx, y + dy

        # bounds check
        if nx < 0 or nx >= MAP_ROWS or ny < 0 or ny >= MAP_COLS:
            continue

        # hit mine → eliminated
        if rover_land[nx][ny] == 1:
            alive = False
            break

        x, y = nx, ny

    path[x][y] = '*'
    executed.append(cmd)

    return {
        "status": "Finished" if alive else "Eliminated",
        "position": {"x": x, "y": y},
        "executed_commands": executed,
        "path": path
    }

# -----
# HELPERS
# -----
def update_map_from_mines():
    for r in range(MAP_ROWS):
        for c in range(MAP_COLS):
            land_map[r][c] = 0
    for m in mines:
        if 0 <= m["x"] < MAP_ROWS and 0 <= m["y"] < MAP_COLS:
            land_map[m["x"]][m["y"]] = 1

# -----
# MAP ENDPOINTS
# -----
def load_map_from_file():
    global land_map, MAP_ROWS, MAP_COLS

    with open("map.txt", "r") as f:

```

```

        lines = f.readlines()

    grid = []
    for line in lines:
        row = list(map(int, line.strip().split()))
        grid.append(row)

    land_map = grid
    MAP_ROWS = len(grid)
    MAP_COLS = len(grid[0]) if grid else 0

@app.get("/map")
def get_map():
    load_map_from_file()
    return {"rows": MAP_ROWS, "cols": MAP_COLS, "data": land_map}

@app.put("/map")
def update_map(rows: int = Query(10), cols: int = Query(10)):
    global MAP_ROWS, MAP_COLS, land_map

    MAP_ROWS, MAP_COLS = rows, cols
    land_map = [[0 for _ in range(cols)] for _ in range(rows)]

    # remove out-of-bounds mines
    mines[:] = [m for m in mines if 0 <= m["x"] < rows and 0 <= m["y"] < cols]

    update_map_from_mines()
    return {"message": f"Map updated to {rows}x{cols}"}

# -----
# MINES
# -----

@app.get("/mines")
def get_mines():
    return mines

@app.get("/mines/{mine_id}")
def get_mine(mine_id: int = Path(..., ge=1)):
    mine = next((m for m in mines if m["id"] == mine_id), None)
    if not mine:
        raise HTTPException(404, "Mine not found")
    return mine

@app.post("/mines", status_code=201)
def create_mine(mine: MineCreate):
    if not (0 <= mine.x < MAP_ROWS and 0 <= mine.y < MAP_COLS):
        raise HTTPException(400, "Mine coordinates out of bounds")

    mine_id = max([m["id"] for m in mines] + [0]) + 1

```

```

    new_mine = {"id": mine_id, "serial": mine.serial, "x": mine.x, "y": mine.y}
    mines.append(new_mine)

    update_map_from_mines()
    return {"id": mine_id}

@app.put("/mines/{mine_id}")
def update_mine(mine_id: int, mine_update: MineUpdate):
    mine = next((m for m in mines if m["id"] == mine_id), None)
    if not mine:
        raise HTTPException(404, "Mine not found")

    if mine_update.serial is not None:
        mine["serial"] = mine_update.serial
    if mine_update.x is not None:
        mine["x"] = mine_update.x
    if mine_update.y is not None:
        mine["y"] = mine_update.y

    if not (0 <= mine["x"] < MAP_ROWS and 0 <= mine["y"] < MAP_COLS):
        raise HTTPException(400, "Mine coordinates out of bounds")

    update_map_from_mines()
    return mine

@app.delete("/mines/{mine_id}")
def delete_mine(mine_id: int):
    mine = next((m for m in mines if m["id"] == mine_id), None)
    if not mine:
        raise HTTPException(404, "Mine not found")

    mines.remove(mine)
    update_map_from_mines()
    return {"message": "Mine deleted"}

# -----
# ROVERS
# -----

@app.get("/rovers")
def get_rovers():
    return [{"id": rid, "status": r["status"]} for rid, r in rovers.items()]

@app.get("/rovers/{rover_id}")
def get_rover(rover_id: int):
    rover = rovers.get(rover_id)
    if not rover:
        raise HTTPException(404, "Rover not found")
    return rover

```

```

@app.post("/rovers", status_code=201)
def create_rover(rover: RoverCreate):
    rover_id = max(rovers.keys(), default=0) + 1
    rovers[rover_id] = {
        "status": "Not Started",
        "commands": rover.commands,
        "position": {"x": 0, "y": 0},
        "executed_commands": []
    }
    return {"id": rover_id}

@app.put("/rovers/{rover_id}")
def set_commands(rover_id: int, rover: RoverCreate):
    if rover_id not in rovers:
        raise HTTPException(404, "Rover not found")

    if rovers[rover_id]["status"] not in ["Not Started", "Finished"]:
        raise HTTPException(400, "Rover is busy")

    rovers[rover_id]["commands"] = rover.commands
    rovers[rover_id]["status"] = "Not Started"
    return {"message": "Commands updated"}

@app.delete("/rovers/{rover_id}")
def delete_rover(rover_id: int):
    if rover_id not in rovers:
        raise HTTPException(404, "Rover not found")

    del rovers[rover_id]
    return {"message": "Rover deleted"}

@app.post("/rovers/{rover_id}/dispatch")
def dispatch_rover(rover_id: int):
    if rover_id not in rovers:
        raise HTTPException(404, "Rover not found")

    rover = rovers[rover_id]

    if rover["status"] != "Not Started":
        raise HTTPException(400, "Can only dispatch Not Started rover")

    result = execute_rover_commands(rover["commands"])

    rover["status"] = result["status"]
    rover["position"] = result["position"]
    rover["executed_commands"] = result["executed_commands"]

    # remove destroyed mines
    update_map_from_mines()

```

```

    return {
        "id": rover_id,
        "status": rover["status"],
        "position": rover["position"],
        "executed_commands": rover["executed_commands"],
        "path": result["path"]
    }

if __name__ == "__main__":
    uvicorn.run(app, host="127.0.0.1", port=8000)

```

server.py

Going to url: <http://127.0.0.1:8000/docs> brings up the menu below, displaying all of the methods implemented in this lab.

default		^
GET	/map Get Map	✓
PUT	/map Update Map	✓
GET	/mines Get Mines	✓
POST	/mines Create Mine	✓
GET	/mines/{mine_id} Get Mine	✓
PUT	/mines/{mine_id} Update Mine	✓
DELETE	/mines/{mine_id} Delete Mine	✓
GET	/rovers Get Rovers	✓
POST	/rovers Create Rover	✓
GET	/rovers/{rover_id} Get Rover	✓
PUT	/rovers/{rover_id} Set Commands	✓
DELETE	/rovers/{rover_id} Delete Rover	✓
POST	/rovers/{rover_id}/dispatch Dispatch Rover	✓



Part 2: Create the operator

This operator application was implemented in Python as a command-line interface that interacts with the FastAPI server using HTTP requests through the requests library. It acts as the client in the system, allowing a user to send commands to the server and display responses in a readable format. The operator provides menu-driven options to manage the map, mines, and rovers, including retrieving the map, creating and modifying mines, and creating, updating, or dispatching rovers. Each operation corresponds directly to a REST endpoint on the server, and all communication is handled in JSON format. Helper functions are used to format and print responses clearly, including a visual representation of the map and the rover's path using a grid format. The operator also includes basic error handling by checking HTTP response statuses and displaying meaningful error messages when requests fail. Overall, this implementation provides a simple but user-friendly interface that enables real-time interaction with the rover demining system and demonstrates client-side control of a RESTful API.

```
import requests
import json

BASE_URL = "http://127.0.0.1:8000"

def print_json(response):
    if isinstance(response, requests.Response):
        if response.ok:
            print(json.dumps(response.json(), indent=2))
        else:
            print(f"Error {response.status_code}: {response.text}")
    else:
        print(json.dumps(response, indent=2))

def print_map(data):
    for row in data["data"]:
        print(' '.join(map(str, row)))

def get_map():
```

```

    resp = requests.get(f"{BASE_URL}/map")
    print("Current Map:")
    if resp.ok:
        print_map(resp.json())
    else:
        print_json(resp)

def update_map(rows=10, cols=10):
    resp = requests.put(f"{BASE_URL}/map", params={"rows": rows, "cols": cols})
    print_json(resp)

def list_mines():
    print_json(requests.get(f"{BASE_URL}/mines"))

def get_mine(id):
    print_json(requests.get(f"{BASE_URL}/mines/{id}"))

def create_mine(serial, x, y):
    print_json(requests.post(f"{BASE_URL}/mines", json={
        "serial": serial, "x": x, "y": y
    })))

def delete_mine(id):
    print_json(requests.delete(f"{BASE_URL}/mines/{id}"))

def update_mine(id, serial=None, x=None, y=None):
    data = {k: v for k, v in {
        "serial": serial, "x": x, "y": y
    }.items() if v is not None}
    print_json(requests.put(f"{BASE_URL}/mines/{id}", json=data))

def list_rovers():
    print_json(requests.get(f"{BASE_URL}/rovers"))

def get_rover(id):
    print_json(requests.get(f"{BASE_URL}/rovers/{id}"))

def create_rover(commands):
    print_json(requests.post(f"{BASE_URL}/rovers", json={
        "commands": commands
    })))

def delete_rover(id):
    print_json(requests.delete(f"{BASE_URL}/rovers/{id}"))

def set_rover_commands(id, commands):
    print_json(requests.put(f"{BASE_URL}/rovers/{id}", json={
        "commands": commands
    })))

```

```

def dispatch_rover(id):
    resp = requests.post(f"{BASE_URL}/rovers/{id}/dispatch")
    print_json(resp)

    if resp.ok and "path" in resp.json():
        print("Rover Path:")
        for row in resp.json()["path"]:
            print(" ".join(row))

def main():
    print("Operator running...")

    while True:
        print("\n1.Map 2.Mines 3.Rovers 4.Exit")
        choice = input("> ")

        if choice == "1":
            get_map()

        elif choice == "2":
            print("l=list c=create d=delete u=update")
            sub = input("> ")

            if sub == "l":
                list_mines()
            elif sub == "c":
                create_mine(input("serial: "), int(input("x: ")), int(input("y:
")))

            elif sub == "d":
                delete_mine(int(input("id: ")))
            elif sub == "u":
                id = int(input("id: "))
                update_mine(id)

        elif choice == "3":
            print("l=list c=create s=set p=dispatch")
            sub = input("> ")

            if sub == "l":
                list_rovers()
            elif sub == "c":
                create_rover(input("commands: "))
            elif sub == "s":
                set_rover_commands(int(input("id: ")), input("commands: "))
            elif sub == "p":
                dispatch_rover(int(input("id: ")))

        elif choice == "4":

```

```

        break

if __name__ == "__main__":
    main()

```

op.py

Results

```

bob@comlab:~/Chanuth_Pathirana_COE892_Lab4-5/rover-api$ uvicorn server:app --reload
INFO: Will watch for changes in these directories: ['/home/bob/Chanuth_Pathirana_COE892_Lab4-5/rover-api']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [4793] using StatReload
INFO: Started server process [4795]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:44832 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:44832 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:44874 - "POST /mines HTTP/1.1" 201 Created
INFO: 127.0.0.1:59322 - "POST /rovers HTTP/1.1" 201 Created

```

uvicorn server:app --reload

Using the above command, I ran the server.py code, and the application startup was complete.

```

bob@comlab:~/Chanuth_Pathirana_COE892_Lab4-5/rover-api$ python3 op.py
Operator running...

1.Map 2.Mines 3.Rovers 4.Exit
> 1
Current Map:
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 5 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 7 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0

```

PUT

/map Update Map

Parameters

Name	Description
rows integer (query)	Default value : 10 10
cols integer (query)	Default value : 10 10

The size of the map can be adjusted through the ui.

```
1.Map 2.Mines 3.Rovers 4.Exit
> 2
l=list c=create d=delete u=update
> c
serial: hello
x: 1
y: 2
{
  "id": 1
}
```

The mine menu has the option to list, create, delete, and update the mine.

```

1.Map 2.Mines 3.Rovers 4.Exit
> 3
l=list c=create s=set p=dispatch
> c
commands: LRLR
{
  "id": 1
}

1.Map 2.Mines 3.Rovers 4.Exit
> 3
l=list c=create s=set p=dispatch
> l
[
  {
    "id": 1,
    "status": "Not Started"
  }
]

```

The rover menu has a list, create, set, and dispatch option.

Part 4: Azure Account Setup

Section 4.1

Home > Microsoft.ContainerRegistry | Overview > COE892Lab42022G05

COE892Lab42022G05 | Access keys ☆ ...

Container registry

Search

- Overview
- Activity log
- Access control (IAM)
- Tags
- Quick start
- Resource visualizer
- Events
- Settings
- Access keys**
- Encryption
- Identity
- Networking
- Microsoft Defender for Cloud

Registry name	COE892Lab42022G05		
Login server	coe892lab42022g05.azurecr.io		
Admin user	☒		
Username	COE892Lab42022G05		
Name	Password	Regenerate	
password		Show	↺
password2		Show	↺

Following the instructions of the lab manual from section 4.1, I was able to access and obtain the information.

Section 4.2

```

FROM python:3.10

WORKDIR /app

COPY . .

RUN pip install fastapi uvicorn

EXPOSE 80

CMD ["uvicorn", "server:app", "--host", "0.0.0.0", "--port", "80"]

```

Dockerfile

In order to do this, I needed to first create a Dockerfile which I pasted above.

```

bob@comlab:~/Chanuth_Pathirana_COE892_Lab4-5/rover-api$ sudo docker build -t rover-api .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   64kB
Step 1/6 : FROM python:3.10
--> 0e36f3b6c9d8
Step 2/6 : WORKDIR /app
--> Using cache
--> 84efca72562d
Step 3/6 : COPY . .
--> Using cache
--> 7c19da502b22
Step 4/6 : RUN pip install fastapi uvicorn
--> Using cache
--> aa2779476632
Step 5/6 : EXPOSE 80
--> Using cache
--> 0b75d181fbf3
Step 6/6 : CMD ["uvicorn", "server:app", "--host", "0.0.0.0", "--port", "80"]
--> Using cache
--> 47c053dbf48c
Successfully built 47c053dbf48c
Successfully tagged rover-api:latest
bob@comlab:~/Chanuth_Pathirana_COE892_Lab4-5/rover-api$

bob@comlab:~/Chanuth_Pathirana_COE892_Lab4-5/rover-api$ sudo docker run -p 8000:80 rover-api
INFO:      Started server process [1]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:80 (Press CTRL+C to quit)
INFO:      172.17.0.1:45630 - "GET /docs HTTP/1.1" 200 OK
INFO:      172.17.0.1:45630 - "GET /openapi.json HTTP/1.1" 200 OK

```

Rover Demining Server 0.1.0 OAS 3.1

/openapi.json

default

GET	/map	Get Map	⌵
PUT	/map	Update Map	⌵
GET	/mines	Get Mines	⌵
POST	/mines	Create Mine	⌵
GET	/mines/{mine_id}	Get Mine	⌵
PUT	/mines/{mine_id}	Update Mine	⌵
DELETE	/mines/{mine_id}	Delete Mine	⌵

```
sudo docker build -t rover-api .  
sudo docker run -p 8000:80 rover-api
```

Next, I had to run the commands above to build the docker image and then I ran it locally using port 80. To open the container, I had to go to this url in the browser: <http://localhost:8000/docs> which brought up the same menu I had from just running the files locally before the docker image was built which meant the docker was set up successfully.

Section 4.3

```
bob@comlab:~/Chanuth_Pathirana_COE892_Lab4-5/rover-api$ sudo docker login coe892lab42022g05.azurecr.io  
Username: COE892Lab42022G05  
Password:  
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.  
Configure a credential helper to remove this warning. See  
https://docs.docker.com/engine/reference/commandline/login/#credentials-store  
Login Succeeded
```

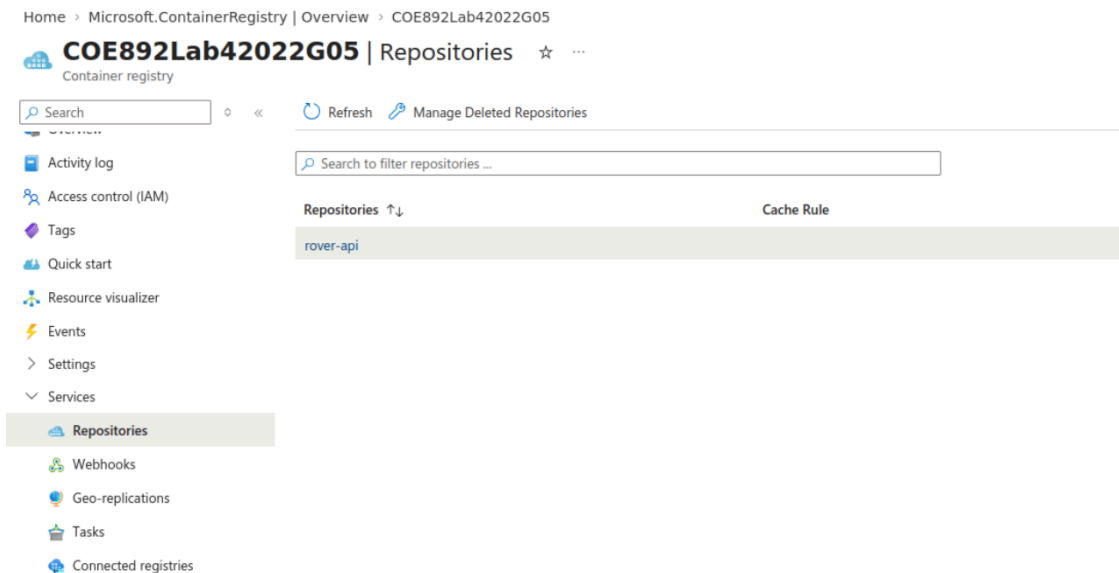
```
sudo docker login coe892lab42022g05.azurecr.io
```

After running the command above, I was able to successfully login by entering the username and password.

```
bob@comlab:~/Chanuth_Pathirana_COE892_Lab4-5/rover-api$ sudo docker tag rover-api coe892lab42022g05.azurecr.io/rover-api:v1  
bob@comlab:~/Chanuth_Pathirana_COE892_Lab4-5/rover-api$ sudo docker push coe892lab42022g05.azurecr.io/rover-api:v1  
The push refers to repository [coe892lab42022g05.azurecr.io/rover-api]  
7e40f6d95524: Pushed  
7d013d9b03c8: Pushed  
72f71fba8171: Pushed  
c133d2fe7445: Pushed  
7fe682552c2a: Pushed  
bc7d147c4fff: Pushed  
6afcf3ecd04: Pushed  
817e939a94eb: Pushed  
dd6e353abeff: Pushed  
c5864b4cf4c9: Pushed  
v1: digest: sha256:82b7f8ee88b436c712c8a94df3b6689ca0812867a9bcd90c54066b926efe8c85 size: 2423
```

```
sudo docker tag rover-api coe892lab42022g05.azurecr.io/rover-api:v1
```

sudo docker push coe892lab42022g05.azurecr.io/rover-api:v1
I used the docker push command to push the container.



As a result, going back to the created COE892Lab42022G05 resource page on Azure and going to the repositories, the upload was successful as shown here.

Section 4.4

Create Web App

Subscription * ⓘ Azure for Students

Resource Group * ⓘ Lab4
[Create new](#)

Instance Details

Name COE892Lab42022G05 ✓
-hbfkfeaddnhnfy7.canadacentral-01.azurewebsites.net

☒ Secure unique default hostname on. [More about this update](#)

Publish * ☐ Code ☒ Container

Operating System * ☒ Linux ☐ Windows

Region * Canada Central

Not finding your App Service Plan? Try a different region or select your App Service Environment.

[Review + create](#) [< Previous](#) [Next: Database >](#)

Create Web App ...

Image Source *

☐ Quickstart
☒ Azure Container Registry
☐ Other container registries

Azure container registry options

Registry *

COE892Lab42022G05

Authentication *

☐ Managed identity
☒ Admin credentials

Image *

rover-api

Tag *

v1

Startup Command ⓘ

Example: /bin/bash; -c; echo hello; sleep 10000

[Review + create](#)[< Previous](#)[Next : Networking >](#)

Here is the link for Docker:

<https://coe892lab42022g05-hbfkfeaddnhnfya7.canadacentral-01.azurewebsites.net>

<https://coe892lab42022g05-hbfkfeaddnhnfya7.canadacentral-01.azurewebsites.net/docs>

Conclusion

In conclusion, I was able to implement the server.py and operator.py successfully by following the lab manual, and ultimately did most of the Azure account setup successfully. These labs all leading up to this one product has taught me more about the steps needed to start from one phase and eventually work my way to developing a bigger product. Overall, these labs have taught me a lot about distributed systems and cloud computing principles that will be beneficial for future endeavours.